

Aula 13 — Como saber se ainda funciona?

Como verificar automaticamente que uma mudança preservou comportamentos que já funcionavam?

Queremos aceitar outras entregas em Pedido

```
pedido.fechar(new EntregaNormal());
```

Antes de mudar essa colaboração, queremos preservar:

- o total dos itens;
- o bloqueio de inclusões após fechar;
- o custo normal de R\$ 10.

Vamos começar pela primeira expectativa e transformá-la em código.

ATIVIDADE

A PRIMEIRA REGRA QUE QUEREMOS PROTEGER

Duas unidades de teclado devem somar quanto?

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
  
System.out.println(pedido.calcularTotal());
```

Qual valor esperamos? Se aparecer outro valor, quem percebe a diferença?

A EXPECTATIVA PRECISA APARECER NO PROGRAMA

Podemos escrever a comparação

```
System.out.println(pedido.calcularTotal() == 300.0);
```

```
true
```

O programa já compara com `300.0`.

Mas esta linha só informa `true` ou `false`; não identifica a regra que falhou.

TORNE A DIVERGÊNCIA IDENTIFICÁVEL

Informe o esperado e o observado

```
double esperado = 300.0;  
double observado = pedido.calcularTotal();  
  
if (observado != esperado) {  
    System.out.println("ERRO no total: esperado " + esperado  
        + ", obtido " + observado);  
}
```

A mensagem identifica esta verificação e os valores diferentes.

ATIVIDADE

LEIA A VERIFICAÇÃO QUE ACABAMOS DE ESCREVER

O que ela informa em cada caso?

O preço continua 150.0 e a quantidade continua 2 .

1. O cálculo devolve 300.0 : o bloco de erro executa?
2. Um defeito faz o cálculo devolver 150.0 : qual mensagem aparece?

UMA COMPARAÇÃO DETECTA UMA DIFERENÇA

Já sabemos verificar esta regra

Resultado do cálculo	Resultado da verificação
300.0	nenhuma mensagem de erro
150.0	esperado 300.0 , obtido 150.0

Agora queremos verificar também remoção, alteração e fechamento.

Como executar todas essas verificações e identificar quais falharam?

A ESTRUTURA QUE COMEÇARIA A SE REPETIR

Cada regra precisa de cenário e expectativa

Para trinta regras, repetiríamos preparação, comparação e mensagens.

Queremos executar o conjunto e receber um resultado para **cada teste**.

JUnit oferece essa organização. Vamos usar o mesmo cenário do teclado.

O primeiro teste protege o total

```
@Test
void calculaTotalDeUmItem() {
    Produto teclado = new Produto("Teclado", 150.0);
    Pedido pedido = new Pedido();
    pedido.adicionarItem(teclado, 2);

    assertEquals(300.0, pedido.calcularTotal());
}
```

A expectativa continua `300.0` ; JUnit registra sucesso ou falha deste método.

Java em foco — JUnit básico

`@Test` identifica um método que o executor deve executar.

`assertEquals(esperado, observado)` verifica a igualdade dos valores.

Se o total vier `150.0`, o teste falha e informa a diferença para `300.0`.

ONDE ESCREVER E EXECUTAR O TESTE

Uma classe de testes, com os imports de JUnit

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class PedidoTest {
    @Test
    void calculaTotalDeUmItem() {
        // corpo do teste mostrado no slide anterior
    }
}
```

Execute a classe pelo executor de testes da IDE.

ATIVIDADE

IDENTIFIQUE O QUE O TESTE FAZ

Prepare → execute → verifique

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
  
double total = pedido.calcularTotal();  
  
assertEquals(300.0, total);
```

Quais linhas preparam o cenário, executam o cálculo e verificam a expectativa?

UM NOME TAMBÉM INFORMA

Qual teste ajuda a próxima pessoa?

```
void teste1() { ... }
```

```
void pedidoFechadoNaoAceitaNovosItens() { ... }
```

O segundo nome indica o comportamento a preservar.

ATIVIDADE

ESCOLHA UMA HISTÓRIA

Como demonstrar essa regra?

| Pedido fechado não aceita novos itens.

Que estado inicial precisamos montar?

Que chamada tenta violar a regra?

Qual observação pública mostraria o resultado?

UM CENÁRIO QUE DISTINGUE RESULTADOS

O pedido fechado preserva o total

```
Produto teclado = new Produto("Teclado", 150.0);  
Produto mouse = new Produto("Mouse", 80.0);  
Pedido pedido = new Pedido();  
  
pedido.adicionarItem(teclado, 1);  
pedido.fechar(new EntregaNormal());  
pedido.adicionarItem(mouse, 1);  
  
assertEquals(150.0, pedido.calcularTotal());
```

O TESTE VÊ O COMPORTAMENTO

Precisamos abrir Pedido por dentro?

O teste não lê:

- o campo `fechado` ;
- a lista privada;
- um índice ou um `if` .

Ele observa `calcularTotal()` depois da tentativa de inclusão.

ATIVIDADE

SEPARE REGRA E IMPLEMENTAÇÃO

Quando esse teste deveria quebrar?

1. Se `Pedido` trocar a forma de percorrer os itens?
2. Se aceitar o mouse mesmo depois do fechamento?
3. Ele também verifica remoção após fechar?

UMA CHAMADA SEM MUDANÇA TAMBÉM IMPORTA

Quantidade negativa preserva o item

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
  
pedido.adicionarItem(teclado, 2);  
pedido.alterarQuantidade(teclado, -1);  
  
assertEquals(300.0, pedido.calcularTotal());
```

A mesma referência de produto identifica a linha.

O EFEITO ESPERADO PODE SER NENHUM

Recusar também é comportamento

A operação foi chamada.

O estado observável deve permanecer igual.

Testar uma regra não exige que o valor final mude.

Prepare dois produtos com o mesmo preço

```
Produto teclado = new Produto("Teclado", 150.0);  
Produto fone = new Produto("Fone", 150.0);  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 1);  
pedido.adicionarItem(fone, 1);
```

O pedido está aberto e contém uma unidade de cada produto.

ATIVIDADE

EXECUTE A REMOÇÃO NO PEDIDO QUE ACABAMOS DE PREPARAR

O total detecta a remoção do produto errado?

```
pedido.removeItem(teclado);  
assertEquals(150.0, pedido.calcularTotal());
```

Se um defeito remover o **fone** em vez do teclado, esta verificação falha?

DOIS ESTADOS PASSAM PELA MESMA COMPARAÇÃO

O total é igual, mas o produto restante é diferente

Depois de removerItem(teclado)	O que restou	Total
implementação correta	um fone	150.0
defeito: remove o fone	um teclado	150.0

`assertEquals(150.0, ...)` passa nos dois casos.

Precisamos de uma observação que distinga **qual produto permaneceu**.

ATIVIDADE

CONTINUE O MESMO CENÁRIO POR UMA OPERAÇÃO PÚBLICA

Tente alterar a quantidade do fone

```
pedido.alterarQuantidade(fone, 2);  
assertEquals(300.0, pedido.calcularTotal());
```

1. Qual total aparece se o fone permaneceu?
2. Qual total aparece se o defeito deixou apenas o teclado?

A SEGUNDA OBSERVAÇÃO DISTINGUE O DEFEITO

Agora o teste percebe qual produto foi removido

Implementação	Após alterar o fone para 2	Verificação
correta	300.0	passa
removeu o fone por engano	150.0	falha

Observamos operações públicas, sem ler a lista privada.

O cenário precisa tornar o defeito que queremos detectar observável.

Proteja o custo que já funciona

```
@Test
void entregaNormalCustaDezReais() {
    Pedido pedido = new Pedido();
    pedido.fechar(new EntregaNormal());

    assertEquals(10.0, pedido.calcularCustoEntrega());
}
```

Quando a dependência mudar, este cenário deve continuar passando.

ANTES DA EXPERIÊNCIA, IDENTIFIQUE A SUÍTE

Vamos executar estes três testes

Método de teste	Expectativa
<code>calculaTotalDeUmItem()</code>	duas unidades somam <code>300.0</code>
<code>pedidoFechadoNaoAceitaNovosItens()</code>	inclusão recusada: total <code>150.0</code>
<code>entregaNormalCustaDezReais()</code>	custo da entrega <code>10.0</code>

Os três cenários foram apresentados nesta aula.

ALTERE SOMENTE A REGRA DA ENTREGA NORMAL

Um defeito em EntregaNormal.java

```
public class EntregaNormal {  
    public double calcularCusto(Pedido pedido) {  
        return 20.0; // deveria continuar sendo 10.0  
    }  
}
```

Produtos, quantidades e expectativas dos testes continuam iguais.

ATIVIDADE

PREVEJA ANTES DE EXECUTAR OS TRÊS TESTES

Quais verificações detectam essa mudança?

1. `calculaTotalDeUmItem()`
2. `pedidoFechadoNaoAceitaNovosItens()`
3. `entregaNormalCustaDezReais()`

Para cada teste, preveja **passa ou falha**.

No teste que falha, indique esperado e observado.

EXECUTE E COMPARE COM A PREVISÃO

A suíte localiza a expectativa que deixou de ser atendida

Verificação	Resultado
total dos itens	passa
inclusão após fechamento	passa
custo da entrega normal	falha: esperado 10.0 , obtido 20.0

Restaure `return 10.0;` em `EntregaNormal` e execute novamente.

Os três testes voltam a passar.

O QUE ESSE VERDE DEMONSTRA?

Uma evidência delimitada

Neste cenário, o comportamento observado corresponde à expectativa codificada.

E se a expectativa estiver errada? E se faltar um cenário importante?

ATIVIDADE

DEFINA O ALCANCE DO TESTE

O que podemos concluir?

1. O teste da entrega normal passa. A entrega expressa funciona?
2. Todos os testes escritos passam. Não há mais bugs?
3. Que regra da Versão 10 ainda vale proteger?

Teste o Projeto 2 que já funciona

- contador inicial e avisos por e-mail;
- sequência com e-mail, SMS e painel;
- dois lembretes independentes;
- uma regra de contagem escolhida por você.

Qual teste você mais gostaria de ter antes de mudar **Lembrete** ?

SÍNTESE

A PERGUNTA AGORA É EXECUTÁVEL

Síntese

- Teste = cenário + operação observada + expectativa.
- Bons nomes explicam a regra protegida.
- Observamos comportamento público, inclusive quando nada deve mudar.
- Passar oferece evidência sobre os cenários escritos.

ANTES DA PRÓXIMA MUDANÇA

Podemos perguntar de novo

Comportamento atual → teste → mudança → teste novamente

Você continua fazendo aquilo que combinamos?